Technical Documentation: Bonded Payables Agent

Competition: 2026 Shenzhen International FinTech Competition (FinTechathon) · International Track

Theme: AI as a Financial Participant | **Topic:** Topic F: Agent Wallet (Autonomous On-chain Wallet)

Deliverable ID: Checklist Item 01 (Technical Documentation) | **Date:** October 20, 2026

Repository: https://github.com/NafisAhnnaf/shenzhen-fintechathon-2026 | **Base Sepolia Verified:** 0x63Ac9033d31DA29EAbc3bD750905E28a781b9844

Technical document

Bonded Payables Agent · FinTechathon 2026, International Track, Topic F (Agent Wallet) 20 Oct 2026. Companion documents: `docs/security_self_assessment.md` (tiers, policies, red team, risks), `docs/spec.md` (every feature with its status), `docs/real_vs_simulated.md`, `docs/erp-adapters.md`, `results/summary.md`, `evidence/`.

An AI accounts-payable agent for Hong Kong manufacturers with 50 to 250 staff. It reads supplier bills and emails, checks them against the company's own records (ERP) and public metal prices, pays routine bills on its own and sends every other bill to the finance manager with a proposed resolution. It writes and sends its own supplier emails, reconciles supplier and bank statements, opens disputes and recovers money it overpaid. The owner and the finance manager can give it requests in plain words; it shows a plan, asks one question when a request has more than one reading, and runs the plan under the same rules. A smart contract enforces every payment rule even if the agent's server is taken over. The owner sets a loss budget β; conformal risk control turns it into how much the agent may pay alone, and a certified bound K(β) sizes the performance reserve behind those payments.

Money moves two ways. Suppliers on the token rail are paid in a settlement token from the company's vault (it stands in for a bank's tokenised deposit, at face value: 1 token is $1). Suppliers on the bank rail are approved and recorded on chain the same way, and paid by bank transfer: the desk writes ISO 20022 pain.001 files for the bank and matches the bank's camt.053 statements, flagging any debit nobody approved.

The company and its documents are invented (no real company's books are available to a hackathon); metal prices (World Bank), exchange rates (ECB), the sanctions list (OFAC), email (AgentMail), storage (Neon Postgres), ERP synchronization (live Odoo 20 SaaS), the public chain (Base Sepolia), and the enterprise consortium chain (FISCO BCOS, with 22/22 benchmark tasks verified) are real. `docs/real_vs_simulated.md` lists every part.

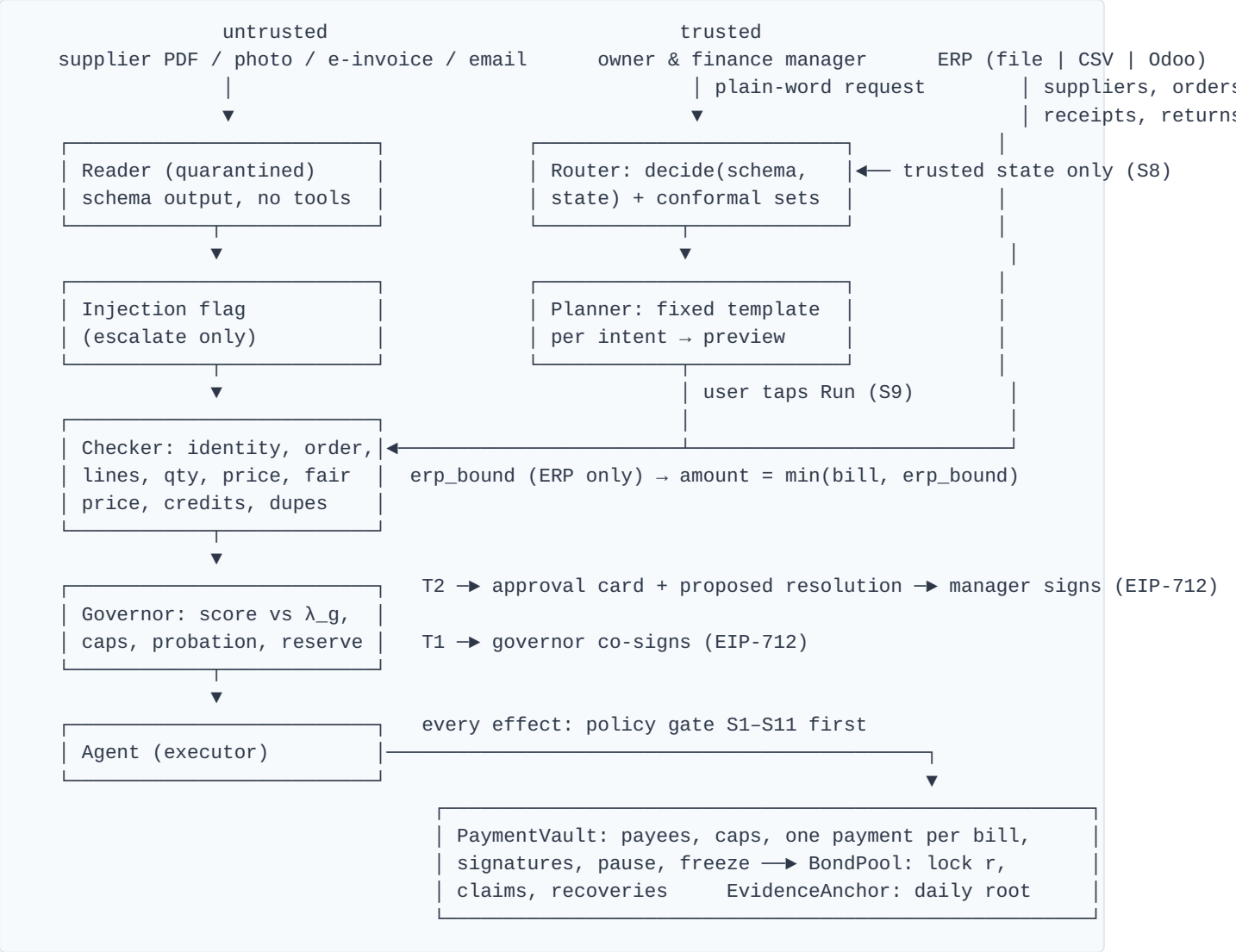
1. What is built and what is not

Area	State	Where
Bill pipeline: intake (e-invoice JSON, PDF and mobile photo through Gemini 3.6 Flash / 3.5 Flash-lite fallback, email attachments), injection flag, checker, governor, T1/T2	Built	`apagent/intake.py`, `reader.py`, `injection.py`, `checker.py`, `governor.py`, `agent.py`
Live reader: upload or mobile photo streams each stage and field bounding box with confidence metrics via SSE (`POST /api/documents/stream`); side-by-side inspection panel	Built	`api.py`, `web/src/components/ReaderPanel.tsx`
Request layer: catalogue, `decide()` backends (rules default; Gemini 3.5 flash-lite opt-in), 60-attack red team (0 unsafe plans), space-evasion normalization, conformal clarification, plan preview / run	Built	`apagent/intent/`, `session.py`, `api.py`
Security layer: capability tags, policies S1–S11, sanctions screening (OFAC SDN), daily payment ceiling, Shadcn UI error boundary	Built, each with a refusing test	`apagent/tags.py`, `policies.py`, `sanctions.py`
Risk core: score, conformal cut-offs, drift guard, probation, K(β), premium	Built	`apagent/risk/`, `governor.py`
Fair-price check and market watch (adaptive conformal ranges, alerts, buy-ahead comparison, PO drafts)	Built	`checker.py`, `marketwatch.py`
Agent actions: emails it sends itself, statement reconciliation, disputes, claims, self-recovery, protective actions, payment-date planner, early-payment offers, budgets, three-way match	Built	`agent.py`, `mail.py`, `views.py`, `budgets.py`
Bank rail: pain.001 payment files, camt.053 statement import and matching, "paid outside the system" alert	Built	`bankfiles.py`
Contracts: `PaymentVault`, `BondPool`, `EvidenceAnchor`, `MockUSDC`	Built; 29 tests; Slither: no high findings	`chain/contracts/`, `chain/test/`, `results/slither.json`
Chains: local Hardhat, local four-node FISCO BCOS, Base Sepolia (deployed and source-verified)	Built	`chain/`, `chain/deployments/`
Public anchoring: every company's journal head, hourly, one Merkle root on Base Sepolia	Built	`public_anchor.py`
Sign-in: email and password, two-step sign-in (TOTP + backup codes), companies, roles (owner, finance manager, clerk, viewer), invitations; wallet sign-in and linking	Built	`auth/` (Better Auth), `identity.py`, `auth.py`
Storage: Postgres (Neon), rows per company, hash-chained append-only journal, signed working memory	Built	`store.py`

Area	State	Where
Email: AgentMail inbox for the agent (send, read, label), supplier sandbox	Built	`mail.py`
ERP adapters: demo file, CSV/Excel with column mapping, live Odoo 17, 18, and Odoo 20 SaaS (XML-RPC)	Built	`apagent/erp/`
Web desk: Dual UI — Executive Purple Desk (`/desk` , `/activity` , `/risk` , `/security` , `/settings`) with animated AI character states, plus scoped Classic Desk (`/bills` , `/approvals` , etc.) and Shadcn UI Alert system	Built; 15+ browser flows	`web/` (React, TypeScript, Vite, wagmi/viem, ECharts)
Local runner: `run_local.sh` single-command orchestrator (Hardhat node, deployments, Better Auth migrations, active readiness probes, Uvicorn)	Built	`run_local.sh`
Hosting: containerized deployment (API, web app, login service, demo chain)	Built	`Dockerfile` , `docker/`
Jev decision model	Interface only (section 4.2)	`apagent/intent/decide.py`
Hosted FISCO with WeBASE	Local multi-node testnet (no cloud instance); Base Sepolia public	`chain/fisco/`

Evidence: the scripted tasks run on a local Hardhat chain, a local FISCO BCOS chain (22/22 tasks completed, including camera photo intake; `evidence/evidence_fisco.md`), and Base Sepolia (`evidence/evidence_baseSepolia.md` , every transaction linked on Basescan). Tests: 155+ Python tests, 29 contract tests, full Playwright E2E browser flows. Slither static analysis: 0 high, 0 medium issues.

2. Architecture



Processes. The FastAPI server (`apagent/api.py`) hosts the agent, the governor and the request layer, and serves the desk; it keeps one desk per company, chosen from the signed-in person's membership. The login service (`auth/` , Better Auth on Node) handles email sign-in, two-step sign-in, companies and invitations; the API forwards `/api/auth/*` to it and reads the session from the shared database. The governor can run as its own service with its own key (`governor_service.py`). With `SIGNING=client` the owner and the finance manager sign in their own wallets and the server checks each signature or mined transaction before acting. State is kept in Postgres (one set of rows per company, the journal hash-chained and append-only); without a database it falls back to a local file.

2.1 Flow 1: every bill

1. **Intake** (`apagent/intake.py` , `Session.ingest_file`). A `.json` e-invoice is read directly. A PDF or photo is read by the vision model; without `VLM_API_KEY` the API answers 503 and records nothing. Emails arrive as RFC 822 text: the sender domain must match a registered supplier exactly, otherwise the email is only logged. Statements arrive as CSV or JSON.
2. **Reader** (`apagent/reader.py`). Document in, typed fields out (`Read` : invoice number, supplier name, tax ID, order, lines, credit, total, bank account, sender domain, per-field confidence, and the free text).

The VLM backend is an OpenAI-compatible call with a strict JSON schema and no tools; output is validated against the schema shape.

3. **Injection flag** (`apagent/injection.py`). Scores the free text after decoding base64, Morse, zero-width and look-alike characters. p ≥ 0.5 adds a T2 reason and a security log line. It can only escalate.
4. **Checker** (`apagent/checker.py` , section 6). Pass or mismatches, margins for the score, `erp_bound` , the payable amount `bounded(bill, erp_bound)` , the fair-price result and a proposed resolution.
5. **Governor** (`apagent/governor.py` , section 7). Duplicate → reject; unknown sender or bank-detail change → hold (and the agent freezes the payee and opens a call-back case). Otherwise T1 if every condition holds, else T2 with reasons.
6. **T1**: policy gate (S1, S2), governor EIP-712 signature over bill, payee, amount, settings ID, expiry and the bill's input hash, then `PaymentVault.payAutonomous` . In reserve modes the vault locks r in the `BondPool` first. A revert turns the bill into T2. **T2**: approval card with evidence and a proposed resolution. The manager signs an EIP-712 `PaymentApproval` for the proposed or an edited amount (never above `erp_bound`); the agent sends `payApproved` .
7. **Record.** The payment is posted back to the ERP (`ErpAdapter.record_payment`); every step is a decision-log line `{ts, actor, kind, planId, inputHash, policy, decision, txHash}` ; each ledger day's Merkle root is anchored on-chain (`Session.close_day` → `EvidenceAnchor.anchor`).

2.2 Flow 2: every request

1. The owner or manager types a request in the request bar (`POST /api/request`).
2. **Router** (`intent/router.py`): checks the state is trusted (S8), then gets intent probabilities from `decide()` , builds the conformal intent set, and for the intent's key slot builds the conformal slot set.
3. **Clarification rule**: one intent and one slot value → plan. Two or three → one question with buttons. More than three, or only UNSUPPORTED → "I can't do that here" plus three example requests. A required slot with no evidence → ask for it with the top candidates. Never more than one question per turn.
4. **Planner** (`intent/planner.py`): fills the fixed template for that intent and returns a preview: every step with bill, supplier, amount, tier and reason, totals and cash after payment.
5. **Run**: steps run in order through the normal bill path. Each payment step re-runs the checker and stops if the bill's status, the payee's freeze or the amount changed since the preview. **Cancel** stops the remaining steps; payments already sent stay sent. Every step writes a log line with the plan ID.

2.3 Backstop modes

Mode	Reserve	On-chain behaviour
Standard	None	`proposeSettings` forces r = 0; `payAutonomous` locks nothing
Self-reserve	Company's own pool deposit	Lock r per T1 payment; a claim executes at once
Bonded	Underwriter's pool deposit	Lock r per T1 payment; bundle needs the underwriter's approval; claims wait out the dispute time; disputes go to the arbiter

The demo runs in bonded mode; task 18 switches to standard mode and back.

3. Data and demo world

`apagent/sim.py` simulates a Hong Kong manufacturer over January to December 2025 with 10 purchased materials and 116 suppliers (the December demo uses 12 of them). Supplier prices for five metals follow real World Bank monthly series (copper, aluminium, zinc, nickel, tin) with a stated pass-through and a one-month lag. Every rate is stated in `sim.RATES` (for example: returns on 12% of bills, 40% of returns recorded in the ERP, 3% prices above tolerance, 1.2% exact and 0.8% fuzzy duplicates, 0.4% bank-change fraud, 0.8% injections, 30% of price changes carry an overcharge of 3–10%). Reader errors follow `reader.ERROR_MODEL` (0.4% per field on a clean scan, ×3 for photos, harder layouts up to ×2.8, credit in a page-2 footer missed 35% of the time).

- The **score** is trained on one simulated year (seed 101).
- The **calibration world** is another year (seed 7): 2,032 bills replayed through the real checker and injection flag (`risk/replay.py`). Groups as the checker assigns them: routine 1,105, price change 379, new supplier 304, other 244. Half is calibration (1,016 bills), half is test; AC-8 repeats the split 100 times.
- The **December demo** (`apagent/demo.py`) is a curated set of documents in `data/demo/documents` (JSON e-invoices plus rendered PDFs and one phone photo), two inbox emails and one supplier statement.

4. Request layer

4.1 Intent catalogue and plan templates

One action per request. Every slot comes from a closed list (`intent/catalogue.py`).

Intent	Key slot (closed list)	Template (`planner.build`)	Control
ASK_STATUS	supplier / material / all (optional)	Select open bills in the window → answer card with total	T0
ASK_WHY	bill (open bills)	Load the bill card and its reasons	T0
WHAT_IF	supplier / material / all (optional)	Select bills → payment-date planner as a simulation → cash after, floor check	T0
PAY_SET	supplier / material / all; window; exclusions (disputed, flagged, probation, above cap)	Select → check and decide each bill (no payment) → skip excluded, rejected and held bills → pay steps with tier per bill → totals, cash after	T1 / T2 per bill
HOLD_SET	supplier / material	Hold each open bill	P
RELEASE	supplier / material	Re-process each held bill through the normal tiers; a fair-price hold never auto-pays	T1 / T2
QUERY_SUPPLIER	bill; topic (price, quantity, credit)	Draft a templated query to the registered contact; send with one tap	T2
TIGHTEN	supplier	Probation for the next 5 bills	P

Intent	Key slot (closed list)	Template (`planner.build`)	Control
SET_DIAL	bundle (careful, standard, more, most)	Show β, T1 share, caps, K and r → prepare the settings transaction → owner signs	T3 (owner only)
PAUSE	—	Pause the agent on-chain	P
RESUME	—	Prepare the unpause transaction → owner signs	T3 (owner only)
UNSUPPORTED	—	Say what it can do, with three examples	—

A request cannot add or change payees, change bank details, move funds out of the vault or pool, pay a supplier not in the registry, or pay more than the ERP-bounded amount: there is no intent for these, so they return UNSUPPORTED.

4.2 `decide()` backends (Jev-ready interface)

All decisions go through `decide(schema, state) -> {option: p}`, which returns a probability for every option in a closed list (`intent/decide.py`).

- `RuleDecider` (default, offline, deterministic): weighted regular-expression and fuzzy-match features per option, softmax over the options; options with no evidence get a floor so the distribution covers the whole list.
- `LLMDecider` (`DECIDER=llm`, `LLM_API_KEY`): any OpenAI-compatible chat model with a strict JSON schema whose keys are exactly the options; probabilities are clipped at zero and renormalised. Default endpoint DashScope (`qwen-plus`).

No Jev backend exists. Jev (TypeSafe AI) would be a third class with the same `decide` method; the conformal sets do not depend on which backend is used, only on its scores on the calibration requests. The supplier-reply classifier (credit note, bank change, other) uses the same interface. The injection flag is a separate deterministic scorer.

4.3 Conformal clarification (KnowNo, Ren et al. 2023)

Calibration. For each calibration request i with true intent y_i, the non-conformity score is s_i = 1 − p(y_i | request_i). With n scores and level α, q̂ is the k-th smallest score with k = ⌈(n + 1)(1 − α)⌉ (if k > n, q̂ = 1 and every option is in the set; `router.qhat`).

Run time. The intent set is C(x) = { y : p(y | x) ≥ 1 − q̂ }.

Guarantee. If the new request and the n calibration requests are exchangeable, the rank of the new score among the n + 1 scores is uniform, so P(s_new ≤ q̂) ≥ k/(n + 1) ≥ 1 − α, which is P(y_true ∈ C(x)) ≥ 1 − α. This holds whatever the decider's own calibration; better probabilities only make the sets smaller. The same rule gives a set for the key slot at level α_S. By the union bound both sets contain the right answer with probability ≥ 1 − α_I − α_S.

Settings and data. α_I = 3% and α_S = 5% (`config.Settings`), so the joint target is ≥ 92% (the plan's 2% slot level was relaxed to 5% in code). 100 requests written and labelled by the team across the 12 intents (`intent/requests_data.py`), split deterministically: 53 calibration, 47 test. Calibrated with the rule decider:

$\hat{q}_I = 0.809$ (an intent enters the set at $p \geq 0.191$) and $\hat{q}_S = 0.082$ (a slot value enters at $p \geq 0.918$) (`data/intent_qhat.json`).

Result on the 47 held-out requests (`results/red.json`, `utility.requests`): intent-set coverage 100%, slot-set coverage 100%, question rate 2%, accuracy when no question is asked 100%, UNSUPPORTED requests refused 100%. Task 20 shows the question: "pay the copper supplier" → "Which supplier?" with ACME, LIONROCK, GANJIANG → the owner taps ACME → plan.

4.4 Preview, run, cancel

- Nothing with an external effect runs before the user taps Run: the policy gate refuses a request-driven effect tool without `plan_confirmed` (S9).
- Task 19: "pay everything from Acme due this week except the disputed bill" → preview → Run → three T1 payments, one bill skipped as disputed.
- Task 21: a five-payment plan cancelled after two payments; steps 3–5 stopped.
- Task 3: "be more careful this month" → preview "Switch to 'careful': loss budget 0.5%, about 14% paid alone, daily cap $10,000 / 3 payments, reserve K $88,363 (r $982 per payment)" → owner signs, underwriter co-signs, new settings active on-chain.

5. Security layer

Full detail, with the enforcing code and refusing test for every policy, is in `docs/security_self_assessment.md`. In short:

- **Capability tags** on every value (`tags.T`); sources `owner`, `settings`, `registry`, `erp`, `computed`, `intent`, `supplier_doc`, `supplier_msg`, `market`. A derived value carries all its sources; any of the last three makes it untrusted.
- **One endorsement**: `bounded(x, erp_bound) = min(x, erp_bound)`, tagged `computed:erp_bounded`; `erp_bound` must be trusted. A supplier can make an amount smaller, never larger.
- **Policies S1–S11** checked before every effect (`policies.check`): payee only from the registry (S1), amount only ERP-bounded (S2), untrusted values only escalate (S3), no payee-changing tool (S4), every card number tagged (S5), no free supplier text in outgoing messages (S6), market values never set payee, amount or quantity (S7), router sees trusted state only (S8), catalogue plans with closed slots and a confirmed preview (S9), protective actions only against the triggering supplier (S10).
- **Permission tiers** T0, P, T1, T2, T3, each enforced in the application and, where money or authority moves, again in the contract.
- **Red team**: 14 attacks; succeeded against the naive agent 13, policies only 2, policies + contract 0.

6. Checker

Deterministic Python (`checker.check`). Inputs: the reader's fields (untrusted) and the ERP records (trusted).

1. **Supplier**: tax ID, else fuzzy name (≥ 0.85); sender domain (look-alike flagged); bank details vs the registry.
2. **Order**: the bill's order number, else rank the supplier's orders by shared items; a tie goes to a person.
3. **Lines**: by item code, else description and unit.
4. **Quantity**: billed ≤ received − already billed (including quantities billed outside the agent, from the ERP).
5. **Price**: within the supplier's tolerance of the order price (1% in the demo). Above it: pay at the order price.
6. **Totals and currency**: lines minus credit equal the total; currency matches.
7. **Credits**: recorded returns not yet used, and open receivables, are deducted. A recorded credit missing from the bill is a mismatch.
8. **Duplicates**: exact invoice number, or a Fellegi–Sunter style agreement score (invoice number similarity +6/−2, same total +7/−4, same order +3/−1, same date +2/−1; ≥ 12 is a duplicate).
9. **Fair price** (section 8).

Amounts. `erp_bound` = Σ over order lines of (received − already billed) × order price × (1 + tolerance), minus recorded credits and open receivables; ERP and settings only. The payable amount from the bill uses each line's quantity capped at what is available and its price capped at the order price (and at the justified price when the fair-price check flags), minus the larger of the bill's credit and the recorded credit, never above the bill's own total, minus open receivables. Then `amount = bounded(payable, erp_bound)`. `test_amount_never_exceeds_erp_bound_on_simulated_year` checks this on every bill of a simulated year.

Proposed resolutions (`checker.propose_resolution`): a decision for each mismatch, built only from typed values: price above tolerance → pay at the order price, hold the difference, draft a query; fair-price excess → pay at the justified price, hold the excess, draft a query; quantity above received → pay for what was received; missing credit → pay net of it; duplicate → reject; bank-detail change → hold, freeze, call back; no order or a tie → hold, manager picks the order.

What matching cannot catch: events nobody recorded (task 4: a return the ERP never saw). Statement reconciliation finds some of these later (section 10).

7. Risk core

7.1 Score

A logistic regression (`risk/calibrate.fit_score`, class-balanced, C = 0.5, standardised features) on typed facts only (`risk/features.py`): 1 − lowest field confidence, 1 − credit-field confidence, price margin, quantity margin, largest fair-price excess, the supplier's confirmed error rate, new item, short tenure, phone photo, log10 of the amount. The target is "this bill would lose money if paid alone". Trained only on eligible bills of the training year (checker passed, 0 < V ≤ P-CAP, not flagged). In the fitted model the largest weights are on the lowest field confidence (0.65), photo (0.25) and amount (0.20).

7.2 Cut-offs by conformal risk control

Loss. For bill i in group g with payable amount V_i ≤ P-CAP, ℓ_i = loss_i / V_i ∈ [0, 1], where loss_i is what would be lost if it were paid alone (`max(0, amount − true amount owed)`, from any covered cause). Define

L_i(λ) = ℓ_i · 1[eligible_i and s_i ≤ λ],

which is non-decreasing in λ and bounded by B = 1. Ineligible bills (checker failed, flagged, above P-CAP) never go T1, so their L_i is 0 for every λ.

Cut-off (Angelopoulos et al., ICLR 2024; `risk/crc.cutoff`). Over the candidate cut-offs Λ = {−1} ∪ {scores},

λ_g(β) = max { λ ∈ Λ : (Σ_{i=1..n_g} L_i(λ) + B) / (n_g + 1) ≤ β }.

λ = −1 means no T1 for the group. The loop stops at the first λ that breaks the bound, which is valid because the left side only grows with λ; tied scores are admitted together.

Guarantee. If a new bill and the n calibration bills of its group are exchangeable, E[L_new(λ_g)] ≤ β. *Proof sketch.* Let λ* be the same rule applied to all n + 1 bills with the true L_new in place of B: λ* = max{ λ : Σ_{i=1..n+1} L_i(λ) / (n + 1) ≤ β }. Because L_new(λ) ≤ B, every λ that passes the calibration rule also passes this one, so λ_g ≤ λ*, and by monotonicity L_new(λ_g) ≤ L_new(λ*)*. λ* is a symmetric function of the n + 1 exchangeable bills, so E[L_new(λ*)] = E[(1/(n + 1)) Σ_i L_i(λ*)] ≤ β.

In words: over all bills in a group (with V ≤ P-CAP), the expected share of bill value lost through autonomous payments is at most β. The live governor adds further conditions (caps, probation, freezes, pause, reserve); each can only remove bills from T1, which only lowers the loss, so the bound still holds.

Data limit. With zero losses the rule still needs B/(n_g + 1) ≤ β, so a group gets any T1 only if n_g ≥ 1/β − 1 (`crc.min_n`): 199 bills at 0.5%, 99 at 1%, 66 at 1.5%, 49 at 2%. The smallest supported β per group in the calibration half: routine 0.29%, price change 0.81%, new supplier 1.06%, other 1.08%. So at β = 1% new-supplier and "other" bills get no T1.

Calibrated cut-offs (`data/calibration.json`, split seed 0):

Bundle	β	routine	price change	new supplier	other	T1 share (calibration)
careful	0.5%	0.350	none	none	none	13.8%
standard	1.0%	0.871	0.307	none	none	47.5%
more	1.5%	0.979	0.985	0.360	0.991	80.5%
most	2.0%	0.979	0.985	0.841	0.991	85.1%

7.3 AC-8: loss ratio vs budget over 100 splits

`apagent/experiments.ac8`, `results/ac8.json`.

Bundle	β	Mean loss ratio (all groups)	Worst group, share of splits over β	Mean T1 share
careful	0.5%	0.130%	routine: 5%	21.9%
standard	1.0%	0.413%	routine: 3%	52.9%

Bundle	β	Mean loss ratio (all groups)	Worst group, share of splits over β	Mean T1 share
more	1.5%	0.586%	new supplier: 1%	72.9%
most	2.0%	0.677%	routine: 0%	83.3%

Baselines (no calibration): person checks all: T1 share 0.0%, loss ratio 0.000%. Pay alone under $5,000: T1 share 84.9%, loss ratio 0.721%. Confidence over 0.95: T1 share 23.8%, loss ratio 0.080%.

Reading: the mean loss ratio stays under β in every bundle. The guarantee is in expectation, so single splits can exceed β (at most 5% of splits in the worst group). At β = 2% the agent pays 83.3% of bills alone at a loss ratio below the "$5,000 rule". The plain confidence threshold has a lower loss but a fixed, low T1 share with no budget behind it. Because calibration and test come from the same simulator, AC-8 checks the implementation more than it tests the method.

7.4 Drift guard, probation, other rules

- **Drift guard** (`Governor.outcome`): λ_g = λ_g^cal + d_g, with d_g ← min(0, d_g + η(β − ℓ_t)) after each known outcome of a T1 bill. η = 0.1051, set so one typical error lowers λ_g by at least one score bin (1/50 of the range). Task 4: one confirmed missed credit lowers the routine cut-off from 0.8715 to 0.8426.
- **Probation**: a confirmed error sends the supplier's next 5 bills to T2.
- **Limit splitting**: if the supplier's rolling 7-day T1 total plus this bill exceeds P-SUPWEEK ($5,000), the bill goes to T2; the contract enforces a weekly cap too (task 12).
- **P-CAP is fixed** across bundles ($5,000), so a larger budget cannot reclassify large payments as routine.
- **Premium** (bonded mode): 1.2 × ê_g × V + r × ρ × 30/365, with ê_g the loss ratio of T1 calibration bills in the group and ρ = 10% a year.
- **Payment-date planner** (`Agent.plan_dates`): pay early only if the discount's annual return d/(1 − d) × 365/(N − D) beats the credit rate (8%) and cash stays above the floor ($150,000); best returns first; nothing moves past its due date. For 2/10 net 30 the return is 37.2% (task 13: discount $47.95 taken). WHAT-IF uses the same planner as a simulation.

7.5 Certified capital K(β)

Conformal risk control bounds the average loss. A reserve also needs a bound on a bad window.

Target. K(β) such that P(total claims on T1 payments open in one 30-day window > K(β)) ≤ ε, with confidence 1 − δ over the calibration sample. ε = 1%, δ = 5%.

Construction (`risk/capital.py` , `risk/calibrate.calibrate_bundles`).

1. For each bundle (m = 4), compute the cut-offs λ_g(β).
2. For calibration bills that would be T1, z = loss / P-CAP ∈ [0, 1].
3. Upper confidence bounds $\bar{\mu} \geq E[z]$ and $\bar{\nu} \geq E[z^2]$ by betting (Waudby-Smith and Ramdas, JRSS-B 2024), each at level δ/(2m). The implementation tests candidate means m upward from the sample mean with the capital process ∏(1 − λ_i(x_i − m)), predictable plug-in bets λ_i truncated at 0.75/(1 − m), and returns the first m at which the capital reaches 1/α (Ville's inequality). `test_betting_upper_bound_covers` checks coverage ≥ 95% on 200 samples.

4. The contract allows at most N = P-NDAY × 30 T1 payments in a window.
5. Bernstein's inequality gives

$$K(\beta) = \min(\text{P-CAP} \times [\, N\bar{\mu} + \sqrt{(2\,N\,\bar{\nu}\ln(1/\varepsilon))} + (2/3)\ln(1/\varepsilon)\,] \,,\ \text{P-DAY} \times 30\,).$$

1. r(β) = K(β)/N is locked per T1 payment; the stored r goes into the settings bundle on-chain.

Derivation. Let $z_1..z_M$ be the T1 payments' loss shares in a window, i.i.d. in [0, 1] with mean μ and variance $\sigma^2 \le E[z^2] = \nu$. One-sided Bernstein: $P(\Sigma z_j - M\mu \ge t) \le \exp(-t^2 / (2(M\sigma^2 + t/3)))$. Setting the right side to ε and solving for t gives $t = L/3 + \sqrt{(L^2/9 + 2M\sigma^2 L)} \le (2/3)L + \sqrt{(2M\sigma^2 L)}$, with $L = \ln(1/\varepsilon)$. Because $z \ge 0$, the sum over $M \le N$ payments is at most the sum over N, so the bound at N covers every window the contract allows. Replacing μ and σ^2 by $\bar{\mu}$ and $\bar{\nu}$ keeps the bound valid on the event that all 2m confidence bounds hold, which has probability $\ge 1 - \delta$ by the union bound. Claims on a payment never exceed the amount paid, and the daily cap limits T1 payments to P-DAY per day, which gives the cap P-DAY × 30.

Chen-style comparison (`capital.chen_style`). Per-action reserves with a union bound (our reading of Chen 2026): each payment reserves the $(1 - \varepsilon/N)$ conformal quantile of its own loss. That quantile is finite only with $n \ge N/\varepsilon - 1$ calibration cases (14,999 for N = 150); with fewer, each payment must reserve its full exposure P-CAP, capped at P-DAY × 30. At our data sizes (89 to 550 T1 calibration bills) the Chen-style reserve therefore equals the maximum exposure in every bundle. It needs no independence assumption; ours does.

7.6 CAPITAL results

`apagent/experiments.capital_exp`, `results/capital.json`. "Windows over K" draws N payments from held-out T1 bills, 20,000 windows.

Bundle	T1 share	N	$\bar{\mu}$	K	r per payment	Chen-style	Chen/K	K / max exposure	Windows over K	Windows over expected-loss rule
careful	13.8%	90	0.0757	$88,363	$982	$300,000	3.4x	29.5%	0.00%	12.2%
standard	47.5%	150	0.0254	$62,241	$415	$450,000	7.2x	13.8%	0.00%	28.0%
more	80.5%	240	0.0174	$63,569	$265	$600,000	9.4x	10.6%	0.00%	61.0%
most	85.1%	300	0.0166	$70,270	$234	$750,000	10.7x	9.4%	0.00%	48.6%

Capital@50 (reserve to release a 50% T1 share, interpolated): ours $62,340, Chen-style $461,268.

Reading: our pooled bound needs 3.4 to 10.7 times less reserve than per-action union-bound reserves. K is not monotone in β: the careful bundle has only 89 T1 calibration bills, so its betting bound $\bar{\mu}$ is loose (0.0757 against a sample mean of 0.0028), which makes K larger than at β = 1%. More data shrinks it. The naive "expected loss only" reserve is exceeded in 12–61% of windows; K was exceeded in none.

Correlated stress case (independence broken): one supplier change hits 30 T1 bills in the same window, all failing together, bounded by the per-supplier cap of $21,429 per window (P-SUPWEEK × 30/7).

Bundle	K	Windows over K, typical error size	Windows over K, total loss on all hit bills	K + one supplier's cap	Windows over it, total loss
careful	$88,363	0.00%	0.00%	$109,792	0.00%
standard	$62,241	0.00%	0.00%	$83,669	0.00%
more	$63,569	0.00%	0.00%	$84,998	0.00%
most	$70,270	0.00%	0.00%	$91,699	0.00%

One supplier failing on every bill stays inside K in these runs because the supplier cap limits the shock. Several suppliers failing together is not covered; then T1 stops when the reserve runs out and the caps limit exposure.

8. Fair-price check

For each invoice line of a material with a public price series (copper, aluminium, zinc, nickel, tin) whose unit price differs from the supplier's last price before this bill's order (`checker.fair_price_check`):

- actual change a = p_invoice / p_last − 1;
- justified change j = s × (I_now / I_last − 1), with s the pass-through share (copper 0.85, aluminium 0.80, zinc 0.80, nickel 0.70, tin 0.60) and index values taken L = 1 month before each date (World Bank monthly, $/t);
- excess e = a − j; justified price = p_last × (1 + j); overcharge = max(0, (p_invoice − justified price) × qty);
- **flag** if e > 2 points and e > the 90th percentile of that supplier's own past excess on the item (when it has at least 5 past price changes).

A flagged line is paid at the justified price (the payable amount uses `min`, so market data can only lower it), the bill goes to T2, and the card shows index values labelled with their series and source. Materials with no public series get no check. Market values never set a payee, amount or quantity on a tool call (S7).

Worked example (task 16, `evidence/evidence_localhost.json`). Acme bills 4 t of copper rod at $13,450.03/t. The last price was $12,339.48/t, so a = +9.0%. Copper (lagged) moved from $10,739.91 to $10,812.03 (+0.67%); with s = 0.85, j = +0.57% and the justified price is $12,409.91/t. Excess 8.43 points; overcharge $4,160.48. The bill is held with "pay at the justified price, hold the excess". The agent drafts the query (`Q-001` , built from these typed values only); the manager sends it with one tap; the supplier's emailed reply is classified `credit_note` and its amount read from the text; the bill is paid at $49,639.64 after the manager signs.

Supplier query and reply (`Agent.draft_query` , `receive_email` , `receive_reply`). The reply must come from the supplier's registered domain and quote the query ID. Classes: credit note with an amount → pay net of it, never more than the current proposed amount; credit note without an amount → manager; bank-detail request or text flagged as instructions → freeze the supplier and open a call-back case; anything else → manager.

FAIR results (`results/fair.json`, calibration world): 124 index-driven price changes, 35 injected overcharges. Detected 29 (83%), missed 6, false flags 5 (5.6% of legitimate rises). Dollars caught $6,626; missed $1,032.

9. Contracts

Solidity ^0.8.11 compiled for the `london` EVM target (the newest FISCO BCOS v3 accepts), OpenZeppelin `EIP712`, `ECDSA`, `SafeERC20`. The same bytecode runs on Hardhat, FISCO BCOS and (by configuration) Base Sepolia.

9.1 PaymentVault

Function	Caller	Rule
`addPayee(id, wallet, rail)`	owner	Payee active only after the cool-down (P-COOL); changing a payee restarts it. Rail: token or bank
`freezePayee(id)`	owner, governor, agent	Immediate
`unfreezePayee(id)`	owner	
`proposeSettings(mode, pDay, pNDay, pSupWeek, r)`	owner	Standard and self-reserve bundles activate at once (standard forces r = 0); bonded bundles wait for the underwriter
`approveSettings(id)` / `activateSettings(id)`	underwriter / owner	Bonded activation needs the underwriter's approval
`payAutonomous(billId, supplierId, amount, settingsId, expiry, inputHash, govSig)`	agent	Reverts unless: not paused; not expired; settings ID is the active bundle; governor EIP-712 signature over all these fields; payee exists, cool-down over, not frozen; bill unused; amount ≤ P-CAP; daily amount and count caps; per-supplier weekly cap; in reserve modes `pool.lock` with the stored r succeeds
`payApproved(billId, supplierId, amount, expiry, sig)`	agent	Approver EIP-712 signature over bill, payee wallet, amount and expiry; payee checks; bill unused. No caps, no reserve; works while paused
`openReceivable(billId, amount, expiry, sig)`	agent	Approver signature; amount ≤ paid; once per bill
`recordOffset(billId, againstBillId, amount)`	agent	≤ open receivable; a later paid bill of the same supplier; if the reserve repaid this bill, the recovered amount goes back to the pool
`pauseAgent()` / `unpauseAgent()`	owner, governor, agent / owner	

Function	Caller	Rule
`withdraw(to, amount)`, `setAgent`, `setGovernor`, `setApprover`, `setPool`	owner	The agent key must differ from owner, governor and approver

Bank rail: the vault applies the same checks, moves the tokens to the owner's treasury address and emits `BankPaymentRecorded` with a bank reference; the bank transfer itself happens off-chain (task 15, mainland payee). Time: the vault and pool read time only through `nowSeconds()`, with a time unit of 1 (EVM) or 1000 (FISCO, whose block timestamps are in milliseconds).

9.2 BondPool (performance reserve)

Function	Caller	Rule
`fund` / `withdraw`	owner or underwriter	Withdraw only up to one's own deposit and only if `balance − amount ≥ lockedTotal`
`lock(billId, amount, r, mode)`	vault	Not paused; `lockedTotal + r + amount ≤ balance` (free capital after the lock covers a total loss of this payment)
`release(billId)`	anyone	After P-WINDOW with no claim
`fileClaim(billId, loss, cause, evidence, expiry, sig)`	anyone, with the approver's EIP-712 signature	Inside the window; loss ≤ amount paid; cause overpayment, duplicate or wrong payee
`dispute` / `resolve(approvedLoss)`	underwriter / arbiter	Bonded mode; approved loss ≤ claimed loss
`executeClaim(billId)`	anyone	After the dispute time (bonded) or at once (self-reserve). Pays min(loss, r + free capital) to the vault only; a shortfall is recorded as owed and pauses the pool
`recordRecovery(billId, amount)`	vault	≤ claim paid − already recovered
`pause` / `unpause`	owner or underwriter	Auto-pause when free capital < P-CAP; unpause needs free capital ≥ P-CAP and nothing owed

Only T1 payments create locks, so T2 payments (approved by a person) are not covered.

9.3 EvidenceAnchor and baselines

`EvidenceAnchor.anchor(period, root)`: the agent key only, one non-zero root per period, append-only. It carries each company's daily decision-log root and, on Base Sepolia, the hourly root over every company's journal head (section 16). `MockUSDC`: the settlement token, 6 decimals, at face value (1 token is $1); it stands in for a bank's tokenised deposit. The no-rules baseline of the RED experiment is a plan in `apagent/naive.py`; it is not deployed.

9.4 Parameters

Name	Demo value	Meaning
P-CAP	$5,000	Per-payment T1 cap, fixed across bundles
P-DAY, P-NDAY	careful $10,000 / 3; standard $15,000 / 5; more $20,000 / 8; most $25,000 / 10	Daily T1 amount and count caps
P-SUPWEEK	$5,000	T1 total per supplier per week
r	K(β)/N per bundle; $414.94 in the deployed standard bonded bundle	Reserve per T1 payment
P-WINDOW	300 s (30 days intended)	Coverage window; K(β) uses 30 days
P-DISPUTE	120 s (5 business days intended)	Underwriter's dispute time
P-COOL	300 s locally, 60 s on FISCO (24 h intended)	Payee cool-down
P-BREAK	3 blocked attempts in 600 s	Circuit breaker
Funding	vault $500,000, pool $72,000	`data/demo/deploy_params.json`
Co-sign line	$25,000	Above it the owner signs too (`DUAL_ABOVE_USD`)

9.5 Invariants

B1–B13 each have one test (`chain/test/invariants.test.js`); 15 further tests cover caps, settings binding, bank rail, standard mode, cool-down, the claim and recovery story, dispute and arbiter, pool too small, shortfall, the and the anchor. 29 tests pass; constructors refuse zero addresses. Slither (0.10.4) reports no high-impact findings; the one medium is an enum comparison it misreads, and the lows are the intended time windows and event order after transfers of our own token (`results/slither.json`). The table is in the security self-assessment, section 6.

10. Agent actions beyond paying

- **Statement reconciliation** (`Agent.reconcile`): each statement line is matched to the ledger. An unmatched charge or credit opens a case. A credit against a paid bill where we paid more than the statement says we owed opens a "missed credit" case. Task 4: Shatin PCB bill paid alone at $4,017.60; the statement shows credit CN-2015 for $1,145.02 against it; the manager confirms by signing the receivable and the claim → receivable on-chain → claim filed with the manager's signature → the reserve repays $1,145.02 → routine cut-off drops → probation for 5 bills.
- **Self-recovery** (task 5): the supplier's next bill goes to T2 (probation) with the open receivable deducted; after payment the agent records the offset on-chain and the recovered amount returns to the pool.
- **Protective actions** (tier P): freeze a payee on a bank-detail change in a bill, email or reply (task 6); pause itself after three blocked attempts (task 14); hold; probation.
- **Claims**: filed by the agent only with the finance manager's EIP-712 signature (`fileClaim` checks it).

11. ERP adapters

The agent reads six things from an ERP and writes one back (`apagent/erp/__init__.py` , `docs/erp-adapters.md`): suppliers (tax ID, contact, bank account or wallet), purchase orders and lines, goods received per line, returns, quantity already billed per line, bills posted per supplier; and it records each payment.

ERP=	Adapter	Source
`file` (default)	`FileAdapter`	The demo ledger; payments appended to a journal file
`csv`	`CsvAdapter`	CSV or Excel exports from any ERP through a column-mapping JSON (`data/erp/odoo_export/mapping.json` maps a real Odoo export); payments written to an outbox CSV
`odoo`	`OdooAdapter`	A live Odoo 17, 18, or Odoo 20 SaaS over XML-RPC (including official Odoo cloud instances). Each payment becomes a posted supplier bill linked to the order lines and a registered payment whose memo carries the transaction hash; bills the agent posts are tagged `agent:<bill id>` so a later sync does not count them twice

`merge_into` refreshes company records from the ERP and keeps the agent's own state (bills handled, receivables, cases, probation, freezes, error counts). ERP records are trusted inputs: they set `erp_bound` . The on-chain payee registry is separate and changes only with the owner key, so an ERP bank-detail edit does not redirect a payment.

Odoo 20 SaaS Compatibility: The `OdooAdapter` (`apagent/erp/odoo.py`) incorporates multi-version compatibility: 1. **Invoice Aggregations**: Resolves modern Odoo `read_group` deprecation by automatically falling back to direct posted invoice queries grouped by partner ID. 2. **Payment Schema Resolution**: Queries `memo` and `payment_reference` dynamically (accommodating Odoo 17 vs. 20 schema differences where `account.payment.ref` was removed) and reconciles across `reconciled_bill_ids` or `reconciled_invoice_ids` . 3. **Resilient Ledger Reset**: `remove_agent_postings()` gracefully clears agent-posted bills and unlinks test payments without affecting human-entered records, with safe error guards preventing ledger desynchronization.

Tests (`tests/test_erp.py`): CSV round trip, Excel and a wrong mapping, merge keeps agent state, billed quantities counted once; live Odoo tests run against both local dockerized Odoo 17 (`erp/odoo/docker-compose.yml`) and live SaaS instances (`https://iut-dhonghorsho.odoo.com`).

12. Stated limits and future work

12.1 Stated limits

- **Simulated company.** Error rates, losses, delays, recovery and overcharge rates are simulated and stated in `sim.py` and `reader.py` . AC-8 and CAPITAL use calibration and test data from the same simulator, so they check the implementation more than they test the method on real bills.

- **Guarantees need exchangeability.** The cut-offs need new bills to resemble calibration bills; the request sets need new requests to resemble the 100 team-written requests (53 calibration, 47 test), which are not real users.
- **The capital bound assumes independent errors.** The stress case covers one supplier failing on every bill; several suppliers failing together is not covered.
- **Small groups get no autonomy.** At β = 1%, new-supplier and "other" bills always go to a person.
- **The vision reader runs on Gemini's free tier**, which rate-limits. Accuracy is measured only on the 29 demo documents: 434 of 435 fields exact (99.8%), 3.7 s each, the phone photo included (`results/reader.json`).
- **Jev needs a paid key.** A `JevDecider` exists behind the same interface but was not used for the evidence; the router runs on the rule decider by default, with Gemini as an opt-in (`DECIDER=llm`).
- **Demo windows are compressed** (seconds instead of days) on every chain; K(β) and r use the real 30-day window.
- **The hosted demo runs on a local chain inside its container**, with fresh contracts per company, because judges' companies must not share caps, a pause switch or test keys on one public deployment. Base Sepolia carries the hourly public anchors and the public evidence run; FISCO BCOS runs locally (no server to host it).
- **The bank rail ends at files.** No bank connection: the company uploads pain.001 to its portal and imports the bank's camt.053. The demo bank on local chains books a file the way a statement would.
- **Security limits** (ERP trust, the S3 exception, the owner key as a single point of failure, contract weekly buckets, older web dependencies) are listed in the security self-assessment, section 10.

12.2 Future work

1. **A bank connection** (host-to-host or a bank API) instead of file upload, and a real tokenised deposit (HKMA Project Ensemble) in place of the settlement token.
2. **Jev backend** behind `decide()`, with a data agreement or self-hosted model.
3. **Hosted FISCO BCOS with WeBASE** as the company's main chain, anchored to Base Sepolia as now.
4. **Separate the governor** onto the bank's or the company's risk infrastructure, one per company; multisig owner.
5. **Correlated capital**: treat each supplier-week as one unit in K(β).
6. Supplier portal for revised invoices, federated calibration across companies, an official fapiao verification provider, persistent chain state for the hosted demo.

13. Reproduce

13.1 Single-command local runner (`run_local.sh`)

```
# Launches all services together with active readiness probes:
# 1. Hardhat node (port 8545) & deploys contracts
# 2. Better Auth login service (port 3001) & runs migrations
# 3. Payables Desk API & Web App (port 8000)
./run_local.sh
```

13.2 Step-by-step reproduction

```
python3.12 -m venv .venv && .venv/bin/pip install -r requirements.txt
(cd chain && npm ci && npx hardhat compile && npx hardhat test)      # 29 contract tests
(cd auth && npm ci) && (cd web && npm ci && npm run build)
.venv/bin/python -m pytest tests                                     # Postgres, chain and Odoo tests ski
(cd chain && npx hardhat node)                                       # separate terminal
(cd chain && npx hardhat run scripts/deploy.js --network localhost)
.venv/bin/python -m apagent.tasks --network localhost                # evidence/
.venv/bin/python -m apagent.experiments --red                        # results/
NETWORK=localhost SIGNING=client .venv/bin/uvicorn apagent.api:app --port 8000
```

With a database and the login service (email sign-in): set `DATABASE_URL` , `BETTER_AUTH_SECRET` , `APP_SECRET` , run `(cd auth && npm run migrate && npm start)` and start the API with `STATE_STORE=postgres` . The container (`docker build -t payables-desk .`) runs all of it; `docker/README.md` lists its settings.

14. Sign-in, companies, and UI architecture

Authentication & Roles. Company staff sign in with email and password through the login service (Better Auth 1.7, `auth/`), with optional two-step sign-in (TOTP, backup codes) and passwords checked against known breaches. A company is a Better Auth organisation; a member's role is owner, finance manager, accounts clerk or viewer, and the owner invites people by email. The API never trusts a role in a request: it checks the session cookie's HMAC with the shared secret, reads the session, membership and role from the database (`identity.py`) and opens that company's desk. A person links the wallet they sign with by signing a one-time message; signatures are still checked by the contract. The bank and the arbiter are outside the company and sign in with their wallets only.

Dual UI Shell. The web application (`web/` , React 18, TypeScript, Vite) integrates two complementary interfaces: 1. **Executive Purple Desk (`/desk`)**: A modern, high-level operational desk featuring animated AI clerk character states (`Characters.tsx` expressing thinking, caution, and celebration), a conversational request bar with instant plan previews, bill inspectors, and dispute claim panels. 2. **Classic Back-Office Suite**: Granular enterprise tables (`/bills` , `/approvals` , `/payments` , `/suppliers` , `/disputes` , `/market` , `/bank` , `/controls`) scoped under `.classic` styles with matching design tokens.

Shadcn UI Alert System. To ensure high reliability, all frontend API operations are wrapped in structured `try ... catch ... finally` blocks. Any server exception (HTTP 500), network disconnection, or policy refusal triggers a floating, accessible Shadcn UI Alert box (`web/src/components/Alert.tsx` , `GlobalAlert.tsx`) with clear contextual error details, automatic 8-second dismiss, and manual close

controls. Form actions and approval dialogs feature inline Shadcn alerts via an upgraded `ErrorLine` primitive.

15. Storage

Postgres (Neon in production). Business records are rows per company (bills, suppliers, cases, receivables, emails, anchors, ERP snapshots); a save sends only the rows that changed. The journal is append-only (a database trigger refuses updates and deletes) and hash-chained: each row stores keccak(previous hash ‖ leaf), so `verify_journal` finds any altered, removed or reordered row. The agent's working memory is one blob signed with an HMAC; an altered blob is refused on load.

16. Public anchors

Once an hour every company's journal head goes into one Merkle root, written to `EvidenceAnchor` on Base Sepolia by the agent key (`public_anchor.py`). The Activity page shows the latest anchor, a Basescan link, and whether the company's rows still produce the anchored head (Merkle proof checked). An hour with no new lines sends nothing.

17. Bank rail

Approved bank-rail payments become an ISO 20022 pain.001.001.09 credit-transfer file (end-to-end id = bill id, creditor account from the registry, SHA-256 of the file in the journal). The bank's camt.053.001.08 statement is matched by end-to-end id, or by amount and creditor when the bank drops the reference. A debit that matches nothing the desk approved is a security event and an owner alert ("paid outside the system"). Statements with a DTD or entity declarations are refused.

18. Hosting

One container (`Dockerfile`): the API with the built web app, the login service, and a local demo chain. Each company gets its own contracts on that chain; after a restart a company whose contracts are gone starts again from the sample records. The browser signs with the public test keys through the API's relay, which forwards a signed transaction only from a signed-in person and only to their own company's contracts. State is in Neon, mail in AgentMail, the hourly anchors on Base Sepolia.

References

Angelopoulos et al., Conformal Risk Control, ICLR 2024. Ren et al., Robots That Ask For Help (KnowNo), CoRL 2023. Waudby-Smith and Ramdas, Estimating means of bounded random variables by betting, JRSS-B 2024. Boucheron, Lugosi and Massart, Concentration Inequalities, OUP 2013. Debenedetti et al., Defeating Prompt Injections by Design (CaMeL), arXiv 2503.18813. Chen, Insuring Every Action, arXiv 2605.25632. Hua et al., Quantifying Trust (Agentic Risk Standard), arXiv 2604.03976. Fellegi and Sunter, A Theory for Record Linkage, JASA 1969. World Bank Commodity Price Data (Pink Sheet), CC BY 4.0.